计算机学院 并行程序设计实验报告

Lab2 SIMD 编程：ANN 近似最近邻搜索

AVX2 / NEON 向量化、量化检索与 FastScan 扩展

柯云超 2413575 计算机科学与技术 2026 年 4 月 28 日

目录

摘要 1

1 问题描述与实验目标 2

2 实验平台、数据集与测量方法 2

3 算法设计与实现 3

4 实验结果与分析 7

5 综合讨论 13

6 结论与后续工作 13

A 开发工具使用说明 14

摘要

本文围绕 ANN（Approximate Nearest Neighbor）选题完成 SIMD 编程实验，在 ARM NEON 与 x86 AVX2 平台上实现并比较串行 Flat、Flat-SIMD、SQ-SIMD、PQ-SIMD 与 4-bit PQ-FastScan。报告重点说明距离计算向量化、两阶段量化检索、PQ 查表累加、SoA/blocking 数据布局、手写 SIMD 与自动向量化差异，并结合 VTune、汇编输出和微基准解释性能瓶颈。实验使用 DEEP100K 数据集 [2]（$N = 100000$，$d = 96$，IP 距离），统一以前 2000 个查询、$k = 10$ 进行评测。

主要结论。 Flat-AVX2 相对串行基线取得 2.60× 加速，但很快受内存带宽限制；SQ-AVX2 在 $p = 100$ 时以近乎无损召回达到 427.01 μs，是低延迟配置中最稳定的一组；PQ-AVX2 与 PQ-FastScan 进一步体现 latency / recall trade-off，其中 FastScan 通过 4-bit LUT 与寄存器内查表显著降低 ADC 粗排开销，在 i9 / 鲲鹏上相对标准 PQ 分别达到最高 4.51× / 1.48× 的加速。整体结果说明，ANN 场景中的 SIMD 优化必须与量化压缩、数据布局和微体系结构特性联合设计。

关键词： ANN，SIMD，AVX2，NEON，标量量化，乘积量化，FastScan，自动向量化

1 问题描述与实验目标

本次 SIMD 编程实验选择 ANN 近似最近邻搜索任务。给定基向量集合

$$\mathcal{B} = \{x_0, x_1, \ldots, x_{N-1}\}, \quad x_i \in \mathbb{R}^{96},$$

以及查询向量 q，实验目标是在 IP（Inner Product）度量下寻找 top-k 最相近的基向量。串行 Flat 的距离定义为

$$d_{\mathrm{IP}}(x, q) = 1 - \sum_{j=0}^{d-1} x_j q_j.$$

直接全扫描的理论复杂度为 $O(Nd)$。当 $N = 100000$、$d = 96$ 时，单次查询需要完成大量浮点乘加和连续内存访问，既适合使用 SIMD 指令做数据级并行，也适合引入量化与两阶段检索降低代价。因此，本文将大问题拆成以下子问题：

1. 对最基础的 Flat 全扫描进行手写 SIMD 并行化；
2. 基于标量量化（SQ）和乘积量化（PQ）构造 “粗排 + 精排” 两阶段检索；
3. 对比不同编程策略，包括内存对齐、软件预取和自动向量化；
4. 在 x86 与 ARM 平台上比较 SIMD 并行化效果；
5. 使用 VTune、编译器向量化报告和汇编输出来分析性能差异的根本原因；
6. 在基础方法之外，引入 4-bit PQ FastScan 作为进一步的结构化优化。

Git 项目链接为：`https://github.com/kyc001/parallel/tree/master/ann`。

2 实验平台、数据集与测量方法

2.1 平台与软件环境

本文主要使用四个平台的数据：Windows i9-13900H（AVX2，主开发与 VTune 平台）、鲲鹏 920 服务器（NEON，正式 ARM 提交平台）、Xeon Platinum 8358 容器（AVX-512，SIMD 宽度梯度补测），以及历史平板 Termux 数据（NEON，保留用于对齐 / 预取 / 规模实验与横向对照）。实验环境如表 2.1 所示。

平台	ISA / SIMD	编译器与工具	主要用途
Windows 11, i9-13900H	x86_64, AVX2 + FMA (256-bit)	MSYS2 MinGW-w64 g++ 14.2.0, VTune 2025	AVX2 实现、P-core 绑核测速、VTune 与汇编分析
鲲鹏 920 服务器	AArch64, NEON asimd / asimddp (128-bit)	openEuler 22.03 (LTS-SP4), GCC/G++ 10.3.1，以 `bash test.sh 1 1` 提交运行	ARM 正式实验数据采集
Xeon Platinum 8358 容器	x86_64, AVX-512F + BW + DQ (512-bit)	Ubuntu 20.04.3 LTS，内核 5.15.0-60-generic，64 核 / 128 线程，g++ -O3 -mavx512f	512-bit SIMD 宽度梯度补测（无 perf 权限）
Android 平板 Termux 历史环境	AArch64, NEON asimd / asimddp	Termux + PRoot Ubuntu	对齐、prefetch、规模实验与历史交叉验证

表 2.1 实验平台与用途

2.2 数据集与评测指标

实验使用 DEEP100K 数据集：100K 条 96 维 float32 基向量、10K 条查询向量，以及对应 top-100 ground truth。所有正式速度实验统一取前 2000 个查询，$k = 10$。性能评价采用平均单查询时延（μs）与 Recall@10。Recall@10 定义为返回的 top-10 中命中 ground truth top-10 的比例。

2.3 测量方式

Windows 平台的主要 AVX2 实验均在单线程下运行；FastScan 结果额外采用 PowerShell 设置处理器亲和性到 P-core 进行校正。鲲鹏结果通过远程 `test.sh` 脚本提交得到。对于自动向量化对比实验，本文单独构造了 96 维点积微基准并重复调用 200 万次，以纳秒级平均耗时评估标量、自动向量化与手写 SIMD 三种版本。

3 算法设计与实现

3.1 总体架构与两阶段检索模型

本文的实现不是把同一个距离函数简单替换成 SIMD 版本，而是按 ANN 检索流水线拆成三层：底层是可复用的 SIMD 距离内核，中层是 Flat / SQ / PQ / FastScan 四类检索结构，上层用统一的 top-p 候选池与 top-k 精排接口评测 latency 和 recall。整体结构如图 3.1 所示。

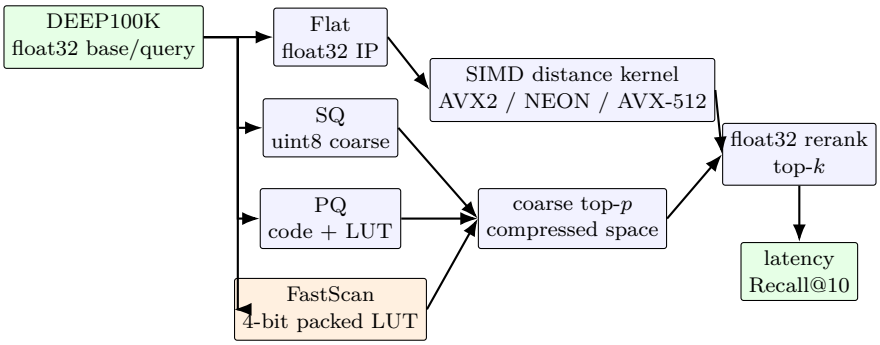


图 3.1 ANN-SIMD 实现架构：Flat 直接复用 float32 SIMD 距离内核；SQ、PQ 与 FastScan 先在压缩空间粗排，再对 top-p 候选使用同一精确内核 rerank。

其中 Flat 是精确全扫描，召回率由 ground truth 决定；SQ、PQ 和 FastScan 都属于两阶段检索。设粗排代价为 C_{coarse}，精排候选数为 p，则单查询时间可概括为

$$T(p) = C_{\text{coarse}} + O(pd).$$

p 越大，精排越接近精确 Flat，recall 越高，但 rerank 时延也越高；p 越小，粗排速度优势越明显，但量化误差更容易把真实近邻排除在候选池外。后文所有 latency-recall 曲线本质上都在观察这个 p 控制的 trade-off。

3.2 串行 Flat 与 Flat-SIMD

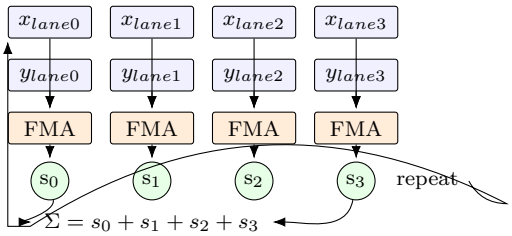
串行 Flat 的核心是对每个基向量执行一次 96 维点积，复杂度为 $O(Nd)$。它完全精确、实现直接，但每个查询都要顺序读取 $N \times d \times 4$ Byte 的原始 float32 数据；在 DEEP100K 上约为 36.6 MB/query，因此很容易从计算瓶颈转向带宽瓶颈。

Flat-SIMD 的设计重点是把点积拆成“向量加载、逐 lane 乘加、延迟归约”三步。AVX2 的 256-bit 寄存器一次处理 8 个 float，NEON 的 128-bit 寄存器一次处理 4 个 float；两者都使用多累加器展开，使连续 FMA 之间不存在单一长依赖链。以 AVX2 为例，本文每轮处理 32 个 float，正好覆盖 96 维向量的 3 个循环块：

Listing 1 Flat-AVX2 的四累加器点积结构（示意）

```
for (; i + 32 <= d; i += 32) {
    sum0 = _mm256_fmadd_ps(load(x + i),      load(y + i),      sum0);
    sum1 = _mm256_fmadd_ps(load(x + i + 8),  load(y + i + 8),  sum1);
    sum2 = _mm256_fmadd_ps(load(x + i + 16), load(y + i + 16), sum2);
    sum3 = _mm256_fmadd_ps(load(x + i + 24), load(y + i + 24), sum3);
}
```

其中 `load(...)` 是 `_mm256_loadu_ps` 的简写。循环内只做独立累加，水平归约推迟到 96 维点积结束之后，避免每 8 个元素就插入 shuffle / add 归约链。理论复杂度仍为 $O(Nd)$，但每周期可发射的浮点工作量显著增加；其上限由 FMA 端口、load 端口和内存带宽共同决定。



32 floats / iter, 4 independent FMAs feeding 4 independent accumulators — reduction is deferred to the end of the loop so the two FMA ports stay saturated.

图 3.2 Flat-AVX2 四累加器点积的数据流：每条 lane 的 FMA 与累加器彼此独立，循环内无跨通道依赖，归约推迟到循环结束后

3.3 SQ-SIMD：标量量化 + 两阶段检索

SQ（Scalar Quantization）对每一维分别统计最小值和最大值，并将 float32 量化到 uint8：

$$\hat{x}_j = \mathrm{clip}\left(\mathrm{round}\left((x_j - v_{\min,j}) \cdot \frac{255}{v_{\max,j} - v_{\min,j}}\right)\right).$$

检索阶段分两步完成：

1. 粗排：将查询同样量化成 uint8，使用 SIMD 计算 96 维 uint8 L2 平方距离，保留 top-p 候选；
2. 精排：仅对 top-p 候选重新计算精确 float32 IP 距离，返回 top-k。

因此 SQ 的复杂度可写为

$$T_{\mathrm{SQ}} = O(Nd_{\mathrm{u8}}) + O(pd_{\mathrm{f32}}),$$

其中第一项由粗排主导，第二项由 rerank 主导。SQ 的关键假设是：每一维的线性量化误差不会显著打乱近邻相对顺序，因此可以先用低精度空间快速筛掉绝大多数向量，再只对少量候选恢复精确 IP。

实现上，粗排内核不再做 float32 FMA，而是对 uint8 差值进行扩展、平方与累加。AVX2 路径将 32 个 uint8 分批扩展到 16-bit，用 `vpmaddwd` 完成“平方 + 相邻对加”，再做向量累加；NEON 路径使用 `vmovl_u8` / `vmlal_u16` 完成同样的数据流。这样 SQ 的 SIMD 收益来自两部分：一是数据宽度从 32 bit 降到 8 bit，cache 中能容纳更多向量；二是整数 SIMD 一次能处理更多 lane。由于本实验数据集中 SQ 的量化误差较小，其粗排顺序与原始 IP 顺序高度一致，因此较小 p 下即可保持很高 recall。

3.4 PQ-SIMD：乘积量化 + ADC 检索

PQ（Product Quantization）将 96 维向量分为 $M = 8$ 个子段，每段维度 $d_{\mathrm{sub}} = 12$。对每个子段分别做 k-means 聚类（本实验取 $k = 256$），编码时每个子段只保存 1 个字节的聚类中心编号，故每个向量仅需 8 字节 [5]。查询时先构建查找表（LUT），再使用 ADC（Asymmetric Distance

Computation）估计距离：

$$\tilde{d}(x,q) = 1 - \sum_{m=1}^{M} \mathrm{LUT}_m[\mathrm{code}_m(x)].$$

PQ 的粗排复杂度约为 $O(NM)$，比 SQ 的 $O(Nd)$ 更低；但由于量化更激进，小 p 时 recall 会明显下降。PQ 的优势在于：粗排极快、内存压缩率高、适合在更大规模数据上扩展。

从数据结构看，PQ 把每个向量分成 M 个子空间，每个子空间用一个独立 codebook 近似原始子向量。离线阶段保存的是 code 而不是 centroid 本身；在线查询时，$\mathrm{LUT}_m[c]$ 表示查询第 m 个子向量与第 c 个 centroid 的距离。这样一次 ADC 扫描只需读取 M 个 code 和 M 个 LUT 项，不再读取 96 个 float。代价是 LUT 索引由 code 决定，访问模式从连续 FMA 变成小表随机查找，后续优化的重点也随之从“算得快”转为“查得快”。

在当前实现中，标准 PQ 的 SIMD 化主要落在两个连续访存的阶段。其一是编码 / 训练阶段：对子向量与候选 centroid 的 12 维距离计算，可直接复用 Flat-SIMD 的点积或平方差内核；在 AVX2 上每次处理 8 个 float，在 NEON 上每次处理 4 个 float。其二是 LUT 构建阶段：`pq_scan_simd.h` 中的 `dot_sub_simd` 直接用于计算 $\mathrm{LUT}_m[c]$。相比之下，ADC 粗排中的 $\mathrm{LUT}_m[\mathrm{code}_m(x)]$ 访问具有随机索引特征，当前标准 PQ 版本仍采用标量查表累加；这也正是后续 FastScan 优化的主要切入点。

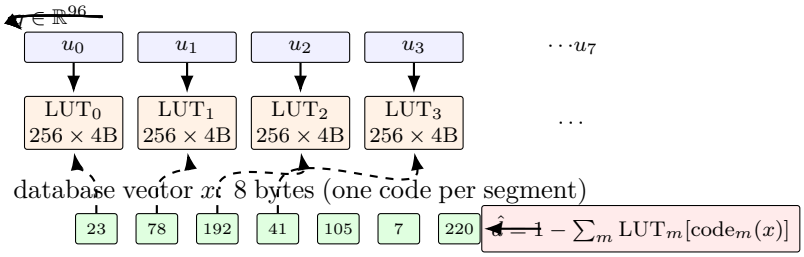


图 3.3 PQ 编码 + ADC 查表示意：查询被切成 $M = 8$ 个 12 维子段 u_m，每段构建一个 256×4 字节的 LUT；数据库向量压成 M 个 1-字节 code，在线扫描时按 code 对每段 LUT 做间接索引并累加。LUT 构建阶段可 SIMD 化，但 code 索引是随机访问（本图右端的虚线），标准 PQ 只能标量累加。

3.5 PQ 编码与 LUT 构建的 SIMD 化设计

PQ 编码阶段在整体时延中占比不大，但从体系结构角度看仍值得说明。若进一步优化，PQ 编码与 LUT 构建都可以统一看作”子向量 × centroid”的小矩阵距离计算。原始 centroid 数据按 AoS 形式存储，即

$$\texttt{centroids}[m][c][j],$$

实现简单且便于直接复用单次距离 SIMD 内核；若追求更高吞吐，可将其转置为 SoA 布局

$$\texttt{centroids_soa}[m][j][c],$$

使得固定查询子向量 $u_m(q)$ 后，AVX2 一次 `_mm256_loadu_ps` 即可取出 8 个 centroid 在同一维度上的值，NEON 则一次取 4 个。此时每个查询维度通过 broadcast 与 FMA 累加，就能把“单 centroid 向量化”进一步提升为“跨 centroid 并行”。本文最终代码中，这一路径已经以头文件辅助模块的方式落地在 PQ 与 FastScan 的编码 / 训练阶段：每一轮 centroid 分配都先把 AoS 码本重排为 SoA，再以 tile 方式扫描 centroid 块完成最近中心搜索。

进一步地，可在子向量批次与 centroid 维度上做 blocking，例如以 32×32 的 tile 组织 (n_{sub}, K_s) 计算，使一个 centroid tile 尽量驻留在 L1 中，减少反复装载。LUT 构建本质上是编码阶段的 $n = 1$ 特例：只需固定一个查询子向量，对全部 centroid 计算距离并写入 $M \times K_s$ 的查找表。因此，跨 centroid 并行、SoA 重排和 tile 化既适用于离线编码，也适用于在线 LUT 构建。考虑到最终查询热路径的常数项，本文在标准 PQ 与 FastScan 的在线 LUT 构建上仍保留了更轻量的单 centroid SIMD / 标量量化实现，而把 SoA + blocking 主要用于编码与训练阶段，从而兼顾结构化优化的覆盖面与端到端查询性能。

上述三条优化可以对应三种不同层面的性能提升路径：AoS→SoA 重排属于*数据布局优化*，32 × 32 tile 化属于 *block + cache* 优化，固定查询子向量、对 8 个 centroid 并行 FMA 属于 *跨 centroid 并行*。三条并列路线在 `pq_blocked_avx2.h` 的 `argmin_l2_blocked` 中统一落地，其核心结构可概括为清单 2。NEON 版本 `pq_blocked_neon.h` 以 `vdupq_n_f32` + `vmlaq_f32` 重现相同拓扑，每轮处理 4 个 centroid。

Listing 2 SoA + blocking 的跨 centroid AVX2 最近中心搜索（要点）

```
for (int block = 0; block < ksub; block += tile) {        // block: cache tile
  for (int c = block; c < block_end; c += 8) {            // c: 8 centroids/lane
    __m256 acc = _mm256_setzero_ps();
    for (int j = 0; j < dsub; ++j) {                       // 固定查询子向量维度
      __m256 xv = _mm256_set1_ps(x[j]);                    // broadcast 1 -> 8
      __m256 cv = _mm256_loadu_ps(soa + j*padded + c);     // SoA: 同维 8 centroid
      __m256 diff = _mm256_sub_ps(cv, xv);
      acc = _mm256_fmadd_ps(diff, diff, acc);              // 跨 centroid FMA
    }
    /* horizontal compare with best_dist ... */
  }
}
```

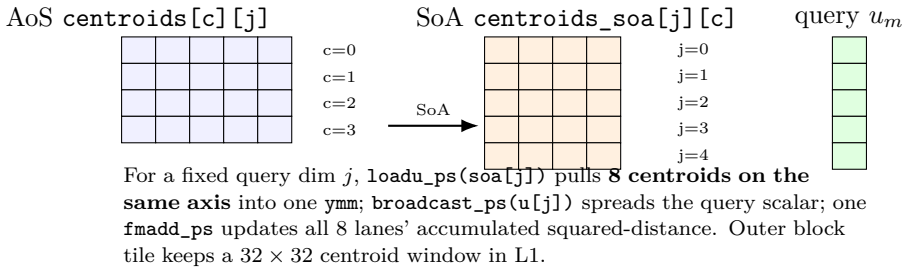


图 3.4 AoS → SoA 数据重排示意：按维度索引后，每次 SIMD 加载取 8 个 centroid 在*同一维度*的值，配合查询 broadcast 做跨 centroid 的并行 FMA

3.6 FastScan 扩展：4-bit PQ 与查表打包

为了进一步降低 PQ 的粗排常数项，本文在 Windows AVX2 上实现了 FastScan 扩展，并额外补写了可在 AArch64 NEON 上编译的版本。该方法直接对应 André 等人在 PQ Fast Scan 中提出的"寄存器内查表"路线 [1]，即 shuffle / fast scan 风格的查表累加。作为延伸阅读，ScaNN 的 AVQ 路线 [4] 与 RaBitQ [3] 也都表明，低比特量化、查表结构重排与 SIMD 友好的数据布局仍是高性能 ANN 的重要方向。本文实现采用 $M = 16$、$K_s = 16$ 的 4-bit PQ，将每两个子段的 code 打包进一个字节，并把 LUT 量化到 0–15 的小整数。需要注意，这并不是在标准 PQ 的 $M = 8, K_s = 256$ 码本上仅替换 ADC 内核，而是在相同 8 B/code 压缩预算下改用另一组 4-bit 码本配置；因此后文与标准 PQ 的比较应理解为两种端到端量化设计的 trade-off 对比，而不是固定码本下只更换查表方式。粗排阶段不再使用 float LUT，而是使用字节查表与饱和累加：

- AVX2 路径使用 `_mm256_shuffle_epi8`；
- NEON 路径使用 `vqtbl1q_u8`。

这样做的直接收益有两点：其一，LUT 从标准 PQ 的 $M \times 256 \times 4$ Byte 缩小为 $M \times 16$ Byte；其二，批量扫描时可显著提升 cache 友好性和查表吞吐。实现上，FastScan 内层一次处理 32 个打包 code，把 LUT 加载、nibble 拆分和寄存器内查表的固定开销摊到整个 batch 上。FastScan 的总复杂度仍可写成

$$T_{\mathrm{FS}} = O(NM) + O(pd),$$

但其中粗排部分的常数显著小于标准 PQ。本文也考虑过使用 gather 指令直接实现标准 ADC 的随机查表：在 x86 上，这一路线会受到随机访问延迟和吞吐不足的限制；而在 AArch64 NEON 上也缺少同等级别、可移植的通用 gather 支持。因此本文最终选择了更符合两平台硬件特点的 shuffle / table-lookup FastScan 路线；4.7 节会给出这一选择的实测依据。

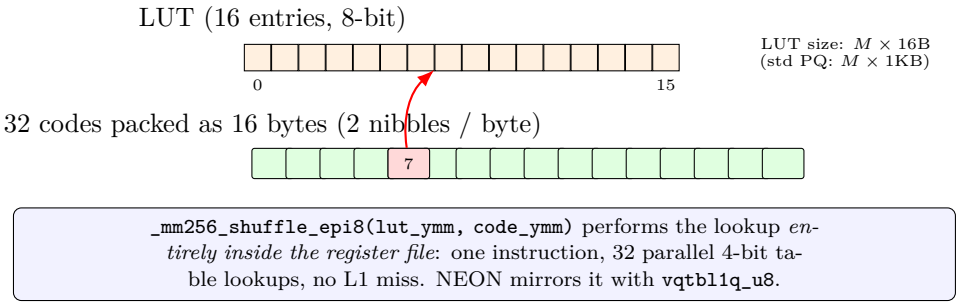


图 3.5 FastScan 的 4-bit 打包 code + 寄存器内 shuffle 查表示意：LUT 缩小到 16 项，码本每 2 段打包为 1 字节，`shuffle_epi8` 在一条指令内完成 32 路 4-bit 查表，绕开了 gather 的随机访存代价

方法	每向量存储	粗排主要代价	备注
Flat / Flat-SIMD	384 B	$O(Nd)$ float32 点积	精确检索
SQ-SIMD	96 B	$O(Nd)$ uint8 L2	4× 压缩，recall 很稳
PQ-SIMD ($M = 8$)	8 B	$O(NM)$ ADC 查表	48× 压缩
FastScan ($M = 16$, 4-bit)	8 B	$O(NM)$ 小 LUT 查表 + 打包累加	LUT 更小；非同码本对比

表 3.2 几种方法的复杂度与存储开销比较

4 实验结果与分析

4.1 核心方法对比：Windows AVX2 与鲲鹏 NEON

表 4.3 汇总了当前最具有代表性的两组核心对比结果：Windows i9-13900H AVX2 与鲲鹏 920 NEON。为了保证 recall 与延迟的平衡，SQ 取 $p = 100$，PQ 与 FastScan 均取 $p = 1000$。

方法	Recall@10	i9 AVX2 (μs)	鲲鹏 NEON (μs)	i9 vs base	鲲鹏 vs base
Baseline Flat	0.99995	4571.62	16120.74	1.00×	1.00×
Flat-SIMD	0.99995	1756.11	5821.16	2.60×	2.77×
SQ-SIMD ($p = 100$)	0.99995	427.01	2422.68	10.71×	6.65×
PQ-SIMD ($p = 1000$)	0.98330	761.99	1984.35	6.00×	8.12×
FastScan ($p = 1000$)	0.97535	541.69	1339.75	8.44×	12.03×

表 4.3 核心方法在 Windows i9 AVX2 (256-bit) 与鲲鹏 920 NEON (128-bit) 上的对比

AVX-512 补测。 Xeon Platinum 8358 上的 Flat-AVX-512 为 1865.11 μs，相对本机串行 7208.06 μs 加速 3.86×。该容器禁止 perf/VTune 采样，因此只作为 SIMD 宽度参考，不与 i9 / 鲲鹏的相对加速直接横比。

核心结论很直接：Flat-SIMD 只降低点积常数，仍受全扫描带宽限制；SQ 在 $p = 100$ 时保持 Recall@10 近乎无损并取得最低时延；PQ 需要较大 p 才恢复 recall；FastScan 在 $p = 1000$ 时同时优于标准 PQ（i9: 761.99 → 541.69 μs；鲲鹏: 1984.35 → 1339.75 μs）。

4.2 SQ / PQ 的时延-召回率权衡

Windows i9 AVX2 的 p-sweep 如图 4.6。SQ 的 recall 基本稳定，延迟主要随 rerank 候选数 p 增长；PQ 更依赖 p，小 p 很快但 recall 明显下降。

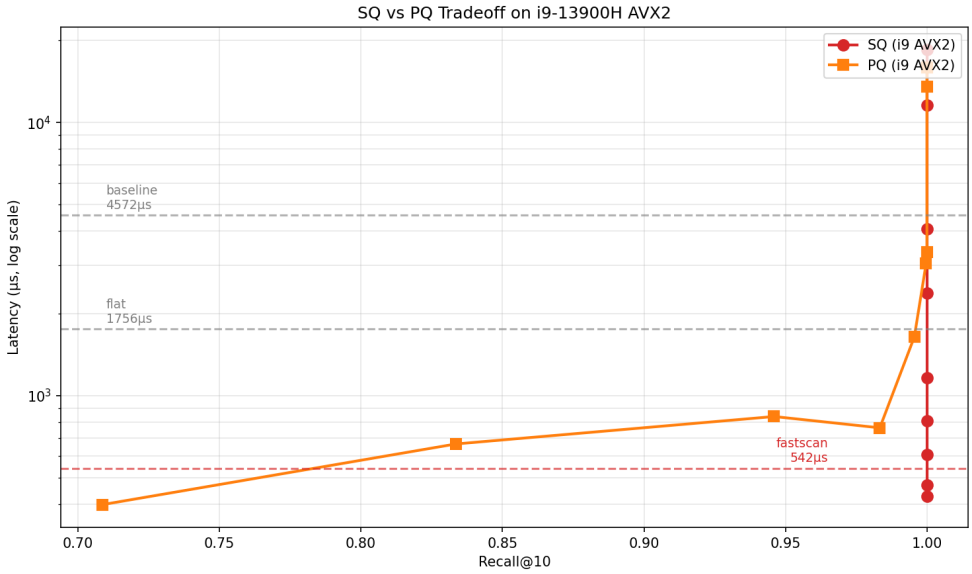


图 4.6 i9-13900H 上 SQ-AVX2 与 PQ-AVX2 的时延-召回率权衡（横轴 = Recall@10，纵轴 = latency）

关键点是：SQ-AVX2 在 $p = 100$ 时为 427.01 μs、Recall@10 = 0.99995；PQ-AVX2 在 $p = 100$ 时虽有 398.53 μs，但 Recall@10 只有 0.70860；提高到 $p = 1000$ 后 PQ 达到 761.99 μs、Recall@10 = 0.98330。三平台曲线见图 4.7，形状基本一致，纵轴差距主要来自 SIMD 宽度、频率和带宽。

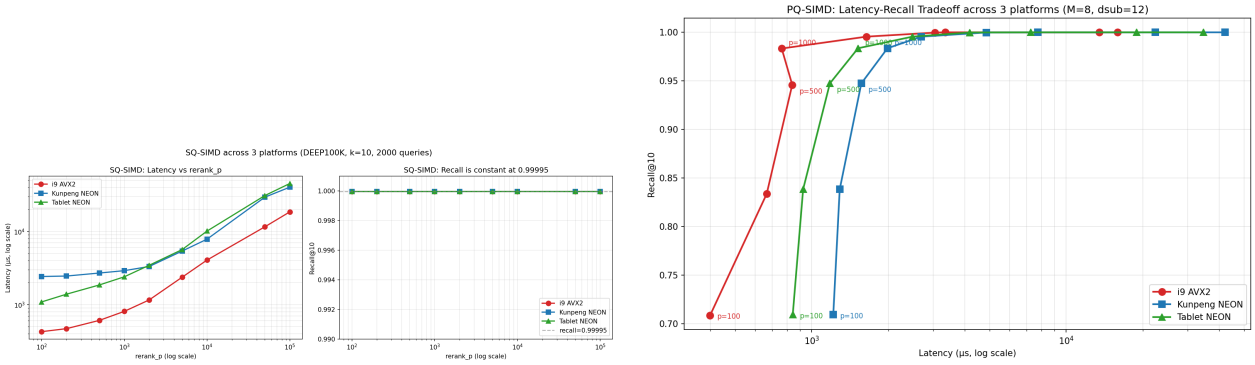


图 4.7 三平台 tradeoff：左 = SQ，右 = PQ。曲线形状跨平台一致；纵轴差距来自 SIMD 宽度 + 频率 + 带宽

4.3 对齐、软件预取与问题规模

本节用三组补充实验覆盖对齐、预取和问题规模。

对齐与非对齐。 E1 使用 64B 对齐、默认 `new[]` 和 1 字节偏移三种分配方式。表 4.4 显示，AArch64 NEON 能执行非对齐访问，但 1B 偏移仍带来约 6.45% 损失，说明 cache line split 仍有代价。

软件预取。 E2 在 Flat-SIMD 流式扫描上尝试多种预取距离。所有显式预取都没有超过无预取基线，说明该访问模式足够顺序，硬件预取器已经覆盖主要需求。

实验项	中位总时延（2000 queries, μs）	相对基线比值（>1 更快）
E1: 64B 对齐	6238505	1.0000×
E1: 默认 `new[]`	6143142	1.0155×
E1: 1B 偏移非对齐	6668860	0.9355×
E2: 无预取	6219560	1.0000×
E2: 预取距离 4	6220804	0.9998×
E2: 预取距离 8	6752636	0.9211×
E2: 预取距离 16	6485157	0.9591×
E2: 预取距离 32	6477509	0.9602×

表 4.4 ARM 历史环境中的对齐 / 预取实验结果

问题规模。 规模实验在 Windows i9-13900H (P-core 绑核) 上取 10K、25K、50K、100K 四个子集，每个规模独立重算 ground truth。结果见表 4.5 与图 4.8。

N	Baseline (μs)	Flat-AVX2 (μs)	SQ ($p = 100$)	PQ ($p = 1000$)
10 000	485.76	104.00	111.09	529.60
25 000	4 096.51	822.34	335.90	1 757.78
50 000	9 620.94	4 066.99	1 592.38	3 457.70
100 000	17 313.80	9 972.43	783.31	4 746.57

表 4.5 Windows i9-13900H 上的规模扫描（单查询平均时延，P-core 绑核，per-N 独立 ground truth）

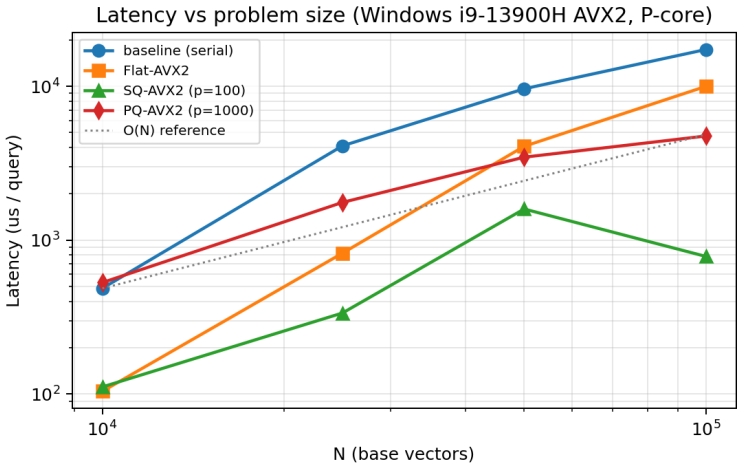


图 4.8 规模实验的 log-log 曲线：Baseline 与 Flat-AVX2 接近 $O(N)$ 线性增长（与灰色参考线平行），SQ / PQ 呈亚线性（rerank 段主导）

Baseline 与 Flat-AVX2 随 N 近似线性增长，符合全扫描 $O(Nd)$；SQ / PQ 在大规模区间开始偏离线性，说明固定 p 后端到端时间逐渐受 $O(pd)$ 的 rerank 段主导。

4.4 手写 SIMD 与自动向量化对比

根据自动向量化与手写 SIMD 对比的研究惯例，本文构造了单独的 96 维点积微基准，对比三种实现：禁用自动向量化的标量版本、允许编译器自动向量化的版本，以及手写 AVX2 版本。表 4.6 给出结果。

版本	单次调用耗时（ns）	相对标量加速比
标量禁向量化	26.7075	1.00×
自动向量化	20.9218	1.28×
手写 AVX2	3.16265	8.44×

表 4.6 Windows i9-13900H 上手写 SIMD 与自动向量化对比

编译器向量化报告中出现了 “loop vectorized using 32 byte vectors”，说明 autovec 确实使用了 256-bit 向量指令；但其加速只有 1.28×。原因是它在循环内做水平归约，形成 `vaddss/vshufps` 长依赖链。手写 AVX2 把归约推迟到循环结束，并使用 4 个独立 FMA 累加器，因此相对 autovec 仍快 6.62×。

版本	反汇编关键形态	性能含义
scalar_novec	`vmulss` + `vaddss`，每轮 1 float	未使用 256-bit lane，吞吐最低
autovec	`vmulps ymm` 后立即插入多条 `vshufps/vaddss`	有向量乘法，但归约链限制 ILP
manual AVX2	4 路 `vfmadd231ps ymm` 独立累加，循环后归约	同时利用 FMA 端口与乱序窗口

表 4.7 手写 SIMD 与自动向量化的内循环形态差异（由 `objdump -Cd -Mintel` 摘要得到）

4.5 VTune 与汇编层面的性能剖析

Windows i9-13900H 上的 VTune 结果进一步解释了三类方法的瓶颈差异。表 4.8 汇总最关键的 Top-Down 指标。

版本	CPI	Retiring	Front-End	Back-End	Memory Bound (⊆ Back-End)	关键特征
Flat-AVX2 (P-core bound)	0.904	19.5%	1.5%	71.4%	56.3%	256-bit FP uOps 33.8%
SQ-AVX2 ($p = 100$, default sched)	0.307	47.5%	5.9%	23.0%	8.7%	256-bit Int SIMD 28.6%
PQ-AVX2 ($p = 1000$, default sched)	0.270	68.5%	8.0%	16.7%	0.6%	3+ ports 85.9%

表 4.8 Windows i9-13900H 上的 VTune 微体系结构结果

需要说明的是：表中 Flat-AVX2 采用 P-core 绑核采样，而 SQ / PQ 取自同一台机器上的默认 Windows 单线程调度样本；因此此表主要用于识别三类方法的瓶颈类型，不直接比较三行的绝对频率或 CPI 高低。

结论集中在瓶颈类型：Flat-AVX2 已明显带宽受限（Memory Bandwidth 占 91.9% 时钟），继续优化 FMA 本身收益有限；SQ-AVX2 同时使用整数 SIMD 粗排和较小 rerank 窗口，是当前最均衡的单点；PQ-AVX2 的 Retiring 比例最高，说明查询阶段较高效，后续主要改 ADC 查表常数。

4.6 SoA + blocking 的编码阶段实测加速

为量化 3.4 节 SoA + blocking 的收益，本文对 100 000 条 12 维子向量、$K_s = 256$ 的一次 k-means `argmin` 分配做微基准。结果如表 4.9。

实现	单次 argmin 全扫 (ms)	加速比
AoS baseline（标量逐 centroid）	151.14 ± 13.71	1.00×
SoA + 32-tile blocking（跨 centroid AVX2 FMA）	43.06 ± 1.92	**3.51**×

表 4.9 PQ 编码阶段 argmin（$N = 100K$，$d_{sub} = 12$，$K_s = 256$）的 AoS 与 SoA+blocking 对比（i9-13900H AVX2，P-core 绑核）

SoA + blocking 加速 3.51×。主要原因是同一维度的 8 个 centroid 可一次装入 SIMD 寄存器，32-tile 又能让 centroid 窗口驻留 L1；同时跨 centroid FMA 消除了单 centroid 标量链的长依赖。

4.7 Gather vs Shuffle：查表累加的路线选择依据

为验证 FastScan 路线选择，本文对 100 000 条 $M = 8$ PQ code 比较三种 ADC 累加方式：标量查表、AVX2 gather、4-bit shuffle。结果见表 4.10。

路线	ns / row	相对标量
scalar（基线）	13.55 ± 0.89	1.00×
gather（`_mm256_i32gather_ps`）	52.73 ± 3.65	**0.26**×（慢 3.9×）
shuffle（FastScan 风格 4-bit LUT）	13.40 ± 3.08	1.01×

表 4.10 ADC 查表累加的三种路线对比（i9-13900H AVX2，P-core 绑核）

结论：AVX2 gather 比标量查表慢 3.9×，不适合这种随机索引 LUT；shuffle 与标量基本持平，但可配合 4-bit 小 LUT、编码打包和 batch 扫描降低端到端常数。因此本文选择 shuffle/table-lookup 路线。

4.8 PQ-FastScan 查表累加优化的实测结果

标准 PQ 的 ADC 查表累加直接决定粗排常数项。FastScan 用寄存器内 4-bit 查表替代随机 LUT 索引，并采用 $M = 16, K_s = 16$ 的 double-nibble code 打包。表 4.11 比较的是标准 PQ（$M = 8, K_s = 256$）与 FastScan（$M = 16, K_s = 16$）两种同为 8 B/code 的端到端量化设计。

p	Recall@10	FastScan (μs)	相对标准 PQ 比值（>1 更快）
40	0.51400	536.447	1.72×
100	0.70075	529.995	0.75×
500	0.93205	526.872	1.59×
1000	0.97535	541.689	1.41×
2000	0.99265	571.850	2.87×
5000	0.99875	674.161	4.51×

表 4.11 Windows i9-13900H 上 PQ-FastScan 的正式结果（P-core 绑核）

在 i9 上，FastScan 从 $p = 40$ 到 $p = 1000$ 时延基本保持在 527–542 μs，Recall@10 从 0.514 提升到 0.975；$p = 5000$ 时达到 0.99875 recall，并把标准 PQ 的 3042.08 μs 降到 674.16 μs。

鲲鹏 920 结果见表 4.12。$p = 1000$ 时 FastScan 为 1339.75 μs、Recall@10 = 0.975253，相对标准 PQ-SIMD 的 1984.35 μs 提升 1.48×；$p = 5000$ 时提升扩大到 2.34×。

p	Recall@10	FastScan (μs)	相对标准 PQ 比值（>1 更快）
500	0.932005	1239.10	1.26×
1000	0.975253	1339.75	1.48×
5000	0.998700	1925.50	2.34×

表 4.12 鲲鹏 920 上 PQ-FastScan 的正式结果（`test.sh` 提交）

图 4.9 给出两平台曲线。高 recall 区间 FastScan 均位于 PQ 左下方，即相同 recall 下时延更低。

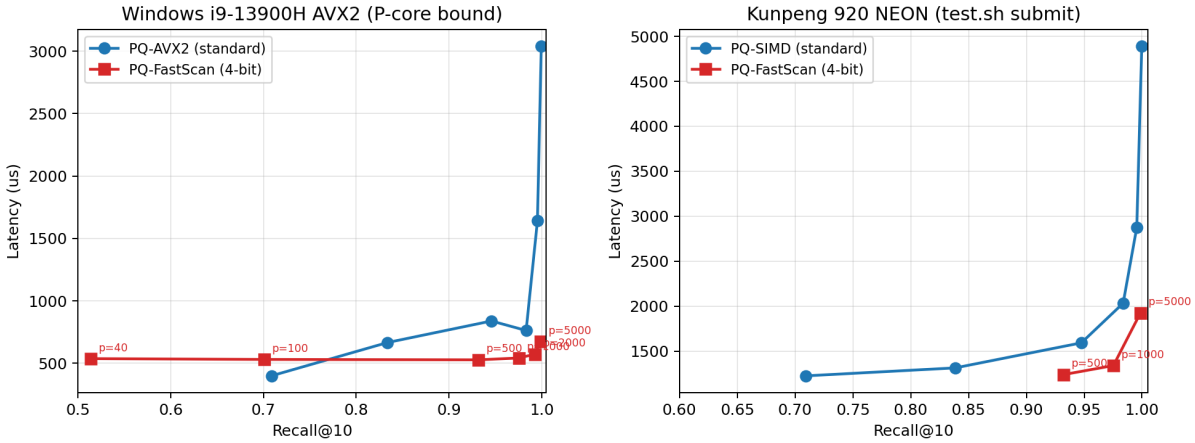


图 4.9 PQ 与 PQ-FastScan 的时延-召回率权衡（横轴 = Recall@10，纵轴 = latency）：左，Windows i9-13900H AVX2（P-core 绑核）；右，鲲鹏 920 NEON（`test.sh` 提交）

Recall@k 敏感性。 考察不同召回深度，本文在 Windows P-core 上补测了 Flat/SQ/PQ/FastScan 四法在 $k \in \{1, 10, 100\}$ 下的 recall，见表 4.13。

方法	Recall@1	Recall@10	Recall@100
Flat-AVX2	1.00000	0.99995	0.99999
SQ-AVX2 ($p = 100$)	1.00000	0.99995	0.94019
PQ-AVX2 ($p = 1000$)	0.99400	0.98315	0.92512
PQ-FastScan ($p = 1000$)	0.99350	0.97525	0.89219

表 4.13 Recall 在不同 top-k 深度下的敏感性

结论：$k = 1$ 时所有方法几乎饱和；$k = 100$ 时量化方法下降明显，说明部署时应把 p 与查询 k 联动调节，通常需保证 p/k 足够大。

4.9 跨平台架构差异分析

跨平台差异主要来自三点。第一，SIMD 宽度不是线性收益：AVX2 为 256 bit，NEON 为 128 bit，AVX-512 为 512 bit，但 Flat-AVX2 已受 DRAM 带宽限制，继续加宽 SIMD 的边际收益下降。第二，单核频率影响很大：i9 P-core 绑核后可达 5.086 GHz，而鲲鹏 920 单核约 2.6 GHz。第三，i9 的 P/E-core 混合调度会引入测量噪声，因此 Windows 关键实验需要绑到 P-core。

FastScan 在 x86 与 ARM 上分别取得 1.41× / 1.48× 的相对 PQ 增益，说明 4-bit LUT、寄存器内查表和打包布局的收益主要来自算法结构，而不是某个平台独有指令。表 4.14 总结了各方法的瓶颈与后续优化方向。

方法	主要瓶颈	证据	对应优化路径	本文对照
Flat-AVX2	DRAM Bandwidth	91.9% of clocks, 42.1 GB/s	量化压缩 / prefetch	SQ/PQ
SQ-AVX2 ($p = 100$)	Balanced	Back-End 23%, Mem-Bound 8.7%	更激进量化	PQ
PQ-AVX2 ($p = 1000$)	Compute / LUT 串行	Retiring 68.5%, 3+ ports 85.9%	shuffle LUT / SoA	FastScan
FastScan-AVX2	LUT + 循环开销	$p = 40 \sim 1000$ 时延近常数	更大 batch / AVX-512	待扩展

表 4.14 三类方法的瓶颈分类、证据与对应优化路径

5 综合讨论

将上述结果合并起来，可以得到三点认识：

1. **Flat-SIMD 受带宽上限限制。** SIMD 降低了点积常数，但全量扫描原始 float32 数据仍需要大量主存带宽。
2. **量化收益主要来自结构变化。** SQ/PQ 把大部分候选提前在压缩空间筛掉，p 成为 latency 与 recall 的核心旋钮。
3. **手写 SIMD 与数据布局仍必要。**自动向量化只能给出基础收益；FastScan、SoA 和 LUT 打包这类优化必须显式重构数据流。

6 结论与后续工作

本文完成了 Flat-SIMD、SQ-SIMD、PQ-SIMD 与 PQ-FastScan 的实现，并在 AVX2、NEON 与 AVX-512 平台上完成 latency / recall 测试。结果显示：SQ-AVX2 ($p = 100$) 是本数据集上最稳健的低延迟配置；标准 PQ 在高 recall 下需要较大 p；FastScan 通过 4-bit LUT、寄存器内查表和打包扫描，在 $p = 1000$ 时相对标准 PQ 于 i9 / 鲲鹏分别提升 1.41× / 1.48×。VTune、反汇编摘要和微基准共同说明，ANN 的 SIMD 优化需要同时考虑指令吞吐、内存带宽、数据布局和量化误差。

后续工作主要是两点：一是在低噪声环境补测更大 p 的 FastScan 曲线；二是在支持 AVX-512 且允许采样的平台上继续验证更宽寄存器查表指令对 FastScan 的收益。

A 开发工具使用说明

本实验在代码调试、脚本拼接和 LaTeX 排版过程中使用过在线大语言模型（豆包）做辅助查询，主要用于 AVX2/NEON intrinsic 参数顺序校对、LaTeX 表格与参考文献格式修正。所有算法设计、参数选择、实验方案、数据采集、结果分析与报告组织均由本人独立完成。以下按 “问题 → AI 回复要点 → 我的判断与修改 → 最终实测结果” 的结构给出一个具有代表性的咨询片段，以说明典型的使用方式。

A.1 举例：PQ 查表累加的优化路线判断

问题背景与我的判断。 在完成标准 PQ 实现之后，通过 VTune 观察到 ADC 粗排阶段存在明显的随机内存访问（$O(NM)$ 次 LUT 间接索引），属于 memory-bound 内核。直接想到的两种 SIMD 改造方向是：(a) 用 `_mm256_i32gather_ps` 把 8 个 centroid 的 LUT 取值合并到一条指令；(b) 把 K_s 从 256 压到 16，用 `_mm256_shuffle_epi8` 做寄存器内查表（FastScan 思路）。根据 Intel 优化手册中 “gather 吞吐随索引散度线性恶化” 的说明，以及 AArch64 NEON 没有等价 gather 的事实，我判断路线 (b) 更可移植、也更契合两平台硬件特点。

AI 回复要点（摘要）。 我向豆包求证的主要是实现细节：$K_s = 16$、$M = 16$ 下 code 以 double-nibble 方式打包成 1 字节后，`shuffle_epi8` 的掩码构造是否会出现 nibble 拆分错位；`adds_epu8` 多层累加下的饱和边界如何设置；以及跨平台头文件 dispatch 的常见写法。工具给出的回复可概括为三点：其一，建议把高/低 nibble 的掩码拆开验证，避免表项错位；其二，建议在每段 LUT 量化时显式记录缩放区间，避免 8-bit 饱和累加过早溢出；其三，建议把 AVX2 / NEON 的 intrinsic 包装在统一头文件下，通过编译期开关选择实现。

我的判断与修改。 这些回复主要提供了 intrinsic 名、掩码常量与头文件组织方式的参考，真正决定路线的仍是我自己的实验判断：我没有采用 gather 路线，而是保留 “$K_s = 16$ + shuffle” 的 FastScan 设计；同时把 LUT 量化比例系数做成每段独立 min/max，把 rerank 阈值改成编译期宏，规避远程作业环境变量丢失，并为 Windows / AArch64 分别补齐可编译的头文件 dispatch。

最终设计与复核。 4.6 节 Gather vs Shuffle 微基准的实测（gather 比 scalar 慢 3.9×、shuffle 与 scalar 持平）以及正式端到端结果都由本人独立完成并复核。最终 i9 与鲲鹏两平台上的 FastScan 相对标准 PQ 分别取得 1.41× / 1.48× 提升（表 4.11、4.12），说明本人的路线判断与后续修改确实转化成了可复现的性能收益。

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proceedings of the VLDB Endowment*, 9(4):288–299, 2015.

[2] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2055–2063, 2016.

[3] Jianyang Gao and Cheng Long. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3), 2024.

[4] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.